

Unit - II User Authentication and Access Control

User authentication and access control make sure only the authorized users reach systems and data, and only to the level they are permitted, which is central to information security

2.1 Identification and Authentication methods:

Identification

- **Identification** is the process by which a user or system asserts an identity to a computer system.
- It **enables the system to recognize** the user by creating a unique profile for access attempts.
- It **initiates the authentication process** by triggering verification challenges like passwords or tokens.
- It **associates future actions** with a specific identity for logging, auditing, and accountability.
- **Identification itself provides no security** because anyone can claim any identity without proof.
- It **only tells the system who the user claims to be**, making no guarantee of legitimacy.
- **Security depends entirely** on the subsequent authentication step to verify the claimed identity.

Example:

Entering a **username**, user ID, or email address during login.

Authentication

- **Authentication** is the process of verifying the claimed identity of a user using electronic credentials or mechanisms.
- Ensures the entity requesting access is legitimate before allowing interaction with system resources, preventing unauthorized users from reaching sensitive data or functions.
- Follows **identification**, where a username claims "who" the user is; authentication then proves "I am who I claim" using passwords, tokens, biometrics, or other verifiers.
- Uses **three main factors** : something you **know** (password/PIN), something you **have** (token/smart card), something you **are** (fingerprint/voice), or something you **do** (typing pattern).
- Protects against **identity spoofing** by binding the claimed identity to proof, blocking attackers who know a username but lack credentials.
- Supports **mutual authentication** (e.g., Kerberos) where both parties verify each other, critical for secure network protocols and session key exchange.

- Implemented via **protocols** like challenge-response, timestamps, or encrypted tokens to defeat replay attacks and ensure timeliness.
 - Forms the **foundation** for access control—without reliable authentication, authorization and auditing lose meaning since actions can't be tied to verified users.
-

1. Electronic user authentication

Electronic user authentication verifies a user's digital identity using electronic credentials before granting access to systems, networks, or data.

Combines **identification** (claiming "who you are" via username) with **authentication** (proving it via credentials) to ensure only legitimate users interact with protected resources

Prevent unauthorized access to systems, networks, or data.

Examples: Logging into a computer, accessing online banking, or entering a secure corporate network.

2. User name and password

Username and password authentication is the **most common and traditional method of user authentication**. It verifies a user's identity based on **something the user knows**. This method is widely used because it is **simple, inexpensive, and easy to implement**.

In this mechanism, the **username identifies the user**, while the **password authenticates the user**.

Working of Username and Password Authentication

The authentication process generally follows these steps:

1. The user enters a **username** and **password**.
2. The system searches for the username in its user database.
3. The entered password is **hashed and compared** with the stored password hash.
4. If both match, the user is **successfully authenticated**.
5. If they do not match, access is **denied**.

Passwords are not stored in plain text but as **hashed or encrypted values** to improve security.

Characteristics

- Knowledge-based authentication
 - Requires no special hardware
 - Can be used for local and remote access
 - Often combined with other methods for better security
-

Advantages

1. **Easy to implement and use**
No additional devices or infrastructure are required.
 2. **Low cost**
Suitable for systems with a large number of users.
 3. **Universally supported**
Works on almost all operating systems and applications.
 4. **Flexible**
Passwords can be changed easily when compromised.
-

Limitations

1. **Weak passwords**
Users often choose easy-to-guess passwords.
 2. **Vulnerable to attacks**
 - Brute force attacks
 - Dictionary attacks
 - Phishing attacks
 - Keylogging
 3. **Password reuse**
Users reuse the same password across multiple systems.
 4. **User dependency**
Security depends heavily on user behavior.
-

Common Attacks on Password-Based Authentication

- **Brute force attack:** Tries all possible combinations
- **Dictionary attack:** Uses common words
- **Phishing:** Tricks users into revealing passwords
- **Shoulder surfing:** Observing password entry
- **Credential stuffing:** Using leaked passwords from other sites

Security Measures to Strengthen Username–Password Authentication

1. **Strong password policy**
 - Minimum length
 - Use of uppercase, lowercase, numbers, symbols
2. **Password hashing and salting**

Prevents password disclosure even if database is compromised.
3. **Account lockout policies**

Lock account after multiple failed attempts.
4. **Regular password changes**

Reduces risk of long-term compromise.
5. **Multi-Factor Authentication (MFA)**

Combine password with OTP or biometrics.
6. **Secure transmission**

Use HTTPS/TLS to prevent interception.

Examples

- Logging into email or social media accounts
- University student portals
- Online shopping websites
- Corporate intranet systems

Flow Diagram: Username and Password Authentication

Start



User enters Username



User enters Password



System checks Username in Database



Is Username Valid?

└─ No → Access Denied



└─ Yes



System hashes entered Password



Compare with Stored Password Hash



Do Passwords Match?

└─ No → Access Denied



└─ Yes



User Authenticated



Access Granted



End

3. Multi-factor authentication

- Requires two or more independent credentials to verify identity.
- Even if a password is stolen, the attacker usually cannot log in without the second factor (e.g., phone code or biometric), which significantly reduces successful account-takeover attacks.
- Greatly reduces risk of compromise if one factor is stolen.

Example: Logging into Gmail with a password + one-time code sent to your phone.

4. Token-based authentic

Definition: Uses a physical or digital token to prove identity.

Types:

Hardware tokens are standalone devices like key fobs or USBs generating codes offline. Software tokens run as apps on phones or computers, relying on the host device's security.

Requires carrying device

Highly portable on mobile

- **Hardware tokens:** Key fobs generating one-time codes.
- **Software tokens:** Mobile apps (Google Authenticator, Authy).
- **Smart cards:** Embedded chips storing credentials.

Strengths: Harder to steal than passwords alone.

Weaknesses: Tokens can be lost, stolen, or cloned.

Example: RSA SecurID token generating time-based codes for corporate VPN access.

2.2 Password attacks:

1. Guessing Password

- **Definition:** Attempting to gain access by repeatedly trying possible passwords until the correct one is found.
- **Methods:**
 - **Brute force:** Trying all possible combinations.
 - **Dictionary attack:** Using common words or leaked password lists.
- **Weakness exploited:** Users often choose simple, predictable passwords.
- **Defense:** Strong password policies, account lockouts after failed attempts, and multi-factor authentication.

2. Piggybacking

- **Definition:** An attacker gains access by following an authorized user into a secure system or physical location.

- **Example:** Sneaking in behind someone who just logged in or entered a restricted area.
- **Weakness exploited:** Lack of monitoring or enforcement of “one person per login/access.”
- **Defense:** Security awareness training, strict access policies, and physical barriers (turnstiles, smart locks).

3. Shoulder Surfing

- **Definition:** Observing someone’s screen or keyboard to capture their password or PIN.
- **Example:** Watching a person type their ATM PIN or login credentials.
- **Weakness exploited:** Human behavior and lack of privacy when entering credentials.
- **Defense:** Privacy screens, awareness of surroundings, and biometric authentication.

4. Dumpster Diving

- **Definition:** Searching through discarded materials (like trash or recycling) to find sensitive information such as written passwords, account details, or confidential documents.
- **Example:** Recovering sticky notes with login details or printed emails from office trash bins.
- **Weakness exploited:** Poor disposal practices of sensitive information.
- **Defense:** Shredding documents, secure disposal policies, and discouraging writing down passwords.

2.3 Biometrics:

Biometric enhanced token - based authentication relies on various physiological or behavioral traits and unique identifiers. Biometric characteristics can be physiological features (for example, a person’s fingerprint, iris, face or hand geometry), or behavioural attributes (such as a person’s gait, signature, or keystroke pattern).

Generally unique to individuals, they can be more effective and reliable at uniquely verifying individuals’ identities than other methods such as knowledge-based verification systems (for example, a password or PIN) or token-based systems (for example, an ID card or licence).

Biometric characteristics cannot be easily shared, lost, or duplicated as passwords or tokens. As such, biometrics are increasingly used in identity management, particularly for the purposes of authentication

Using biometrics for the purpose of authenticating individuals is also known as one-to-one matching.

Examples: fingerprint or facial recognition to access smart phones, or the use of facial recognition technology at airport smart gates.

Steps in Biometric Authentication

1. Enrollment

- The user's biometric trait (e.g., fingerprint, iris, face, voice) is captured for the first time.
- A **template** is created — a digital representation of the trait, not the raw image.
- This template is stored securely in a database or on a smart card.

2. Capture

- During authentication, the system captures a **live sample** of the biometric trait.
- Example: scanning a fingerprint, recording a voice, or taking a facial image.

3. Feature Extraction

- The system processes the live sample to extract key features (patterns, measurements).
- These features are converted into a comparable format (another template).

4. Comparison/Matching

- The live template is compared against the stored template(s).
- Two modes:
 - **Verification (1:1)** → Confirms if the person is who they claim to be.
 - **Identification (1:N)** → Searches across a database to find a match.

5. Decision

- The system decides whether the match is strong enough based on a **threshold**.
- If similarity score $\geq$ threshold → access granted.
- If not → access denied.

Strengths

- Difficult to replicate or steal compared to passwords.
- Provide continuous authentication (e.g., typing rhythm).
- No need to remember complex passwords.
- More reliable for unique identification

Limitations

- Privacy concerns (storage of biometric data) and can be Misuse of collected data by organizations.
 - Environmental factors (lighting, noise, injuries) can affect accuracy.
 - Expensive hardware for advanced biometrics (iris, retina).
 - Cannot be revoked if compromised (you can't change your fingerprint).
 - Biometric data can be spoofed (e.g., deepfakes, fake fingerprints) or stolen from databases.
 - Systems aren't 100% error-free
-

1. Fingerprints

Fingerprint authentication identifies a person based on the **unique ridge and valley patterns** present on fingertips. No two individuals have identical fingerprints, even twins. Fingerprint scanners capture these patterns and convert them into digital templates for comparison. This method is widely used due to its **high accuracy, low cost, and ease of use**, especially in smartphones and access control systems.

Limitations

- Fingerprints can be affected by **cuts, burns, dirt, or moisture**, reducing accuracy
- Some people have **worn or unclear fingerprints** (e.g., manual laborers, elderly)
- Possibility of **spoofing** using artificial fingerprint molds
- Requires physical contact, raising **hygiene concerns**

Example: Smartphones such as Android phones and iPhones use fingerprint sensors to unlock the device and authorize payments. In offices, fingerprint scanners are used for **employee attendance systems**.

2. Hand Prints (Palm Geometry)

Hand print authentication measures the **shape, size, and geometry of the hand**, including palm width, finger length, and curvature. Unlike fingerprints, it does not analyze fine details but focuses on overall hand structure. This method is **fast and reliable** and is commonly used in workplace attendance systems and physical access control.

Limitations

- Less accurate than fingerprint or iris recognition
- Hand shape may change due to **injury, swelling, or weight change**
- Devices are bulky and less suitable for portable systems
- Not ideal for high-security applications

Example: Many factories and construction sites use palm-geometry devices for worker entry and exit recording, ensuring that only authorized employees enter restricted areas.

3. Retina Scan Patterns

Retina scanning uses the **unique pattern of blood vessels at the back of the eye**. A low-intensity infrared light is directed into the eye to capture these patterns. Retina scans offer **very high accuracy and security** because retinal patterns are extremely difficult to duplicate. However, the method is **expensive and intrusive**, limiting its use to high-security environments.

Limitations

- Very **expensive and intrusive**
- Requires the user to look closely into a scanner, causing discomfort
- Not suitable for users with **eye diseases**
- Slow authentication process compared to other biometrics

Iris Scan – Limitations

- High equipment cost
- Performance can be affected by **glasses, contact lenses, or lighting**
- Users may feel discomfort or fear of eye scanning
- Requires user cooperation and stillness

Example: High-security research labs and military facilities use retina scanners to allow access only to authorized scientists or officers, as this method is extremely difficult to forge.

4. Voice Patterns (Voice Recognition)

Voice authentication identifies users based on **speech characteristics** such as pitch, tone, frequency, and speaking style. The system analyzes how a person speaks rather than what they say. Voice recognition is often used in **call centers and remote authentication**, but it can be affected by background noise, illness, or voice changes.

Limitations

- Performance affected by **background noise**
- Voice may change due to **illness, stress, or aging**
- Vulnerable to **voice recording and replay attacks**
- Not suitable for noisy or public places

Example: Banks use voice recognition in customer service call centers, where customers verify their identity by speaking a phrase instead of answering multiple security questions.

5. Face Recognition

Face recognition analyzes **facial features** such as the distance between eyes, nose shape, jawline, and facial contours. Cameras capture images or videos, which are then compared with stored templates. This method allows **contactless authentication** and is widely used in surveillance, smartphones, and public security systems. Its accuracy can be influenced by lighting, facial expressions, or aging.

Limitations

- Accuracy affected by **lighting, camera angle, facial expressions, and aging**
- Can be fooled by **photos, videos, or masks** if anti-spoofing is weak
- Raises **privacy and surveillance concerns**
- Lower accuracy in crowded or outdoor environments

Example: Airports use facial recognition systems for **automated immigration and boarding checks**. Smartphones also use face recognition to unlock devices and secure apps.

6. Signature and Writing Patterns

Signature and handwriting authentication are **behavioral biometrics** that analyze writing style, stroke order, pressure, speed, and rhythm. Unlike simple image matching, advanced systems evaluate **dynamic writing characteristics**. This method is commonly used in banking and legal documentation but may be affected by stress or fatigue.

Limitations

- Writing style may vary due to **fatigue, stress, or health conditions**
- Lower accuracy compared to physical biometrics
- Skilled forgers can imitate signatures
- Slower authentication process

Example: Banks verify customer signatures on **cheques and loan documents** by comparing them with stored signature patterns to detect forgery.

7. Keystroke Dynamics

Keystroke authentication analyzes how a user types, including **typing speed, key pressure, dwell time, and latency between keystrokes**. It is a behavioral biometric that works continuously in the background. Keystroke dynamics enhance security by providing **continuous authentication** but may vary due to mood, injury, or different keyboards.

Limitations

- Typing behavior may change due to **mood, injury, or different keyboards**
- Requires continuous monitoring and learning
- Less accurate for infrequent users
- Not suitable as a standalone authentication method

Example: Online examination systems and corporate networks monitor typing behavior to **continuously verify user identity**, helping detect if someone else is using the account.

<https://www.perplexity.ai/search/token-based-authentic-elaborat-2gq.AR09SNmujUEhBGgilQ#3>

2.4 Authorization

2.4.1 Introduction to Authorization

Authorization is the security process that determines what an authenticated user is allowed to do within a system. After a user's identity is verified through identification and authentication, authorization checks the permissions and privileges assigned to that user before granting access to resources.

Authorization answers the question:

“What actions is this user permitted to perform?”

Authorization controls access to files, databases, applications, network resources, and system functions. Even if a user is successfully authenticated, access is denied if the user does not have the required authorization.

Example:

In a bank system, a teller can view customer account details but cannot modify interest rates, while a bank manager has authorization to approve loans and change account policies.

2.4.2 Goals of Authorization

The primary goals of authorization are to ensure secure, controlled, and appropriate access to system resources. The main goals are explained below:

1. Restrict Unauthorized Access

Authorization ensures that users cannot access resources or perform actions beyond their permissions. This prevents misuse of sensitive information and protects systems from internal threats.

Example: A student can view exam results but cannot modify marks in the university database.

2. Enforce Least Privilege

Authorization follows the principle of least privilege, where users are granted only the minimum permissions required to perform their tasks. This reduces the risk of accidental or intentional damage.

Example: An office employee is allowed to read documents but not delete or modify critical system files.

3. Protect Confidentiality, Integrity, and Availability

Authorization supports the CIA triad by:

- Confidentiality: Preventing access to sensitive data
- Integrity: Allowing only authorized users to modify data
- Availability: Ensuring resources are accessible only to legitimate users

Example: Only authorized doctors can update patient medical records, while others may have read-only access.

4. Support Organizational Roles and Responsibilities

Authorization assigns permissions based on job roles, ensuring that access aligns with organizational structure and policies.

Example: In a company, HR staff can access employee records, while finance staff can access payroll systems.

5. Prevent Insider Threats

By limiting access rights, authorization helps reduce risks from insiders who may misuse their privileges.

Example: A junior employee cannot approve financial transactions, preventing fraud.

6. Enable Accountability and Auditing

Authorization helps track who accessed what and when, enabling accountability and supporting audits and compliance requirements.

Example: System logs record which administrator changed system configurations.

7. Support Secure Multi-User Environments

In shared systems, authorization ensures that multiple users can work simultaneously without interfering with each other's data or privileges.

Example: In a cloud platform, each user can access only their own virtual machines and storage.

2.5 Access Controls

Access control is a fundamental concept in computer and information security. It ensures that **only authorized users can access specific system resources** and perform permitted actions. Access control works together with **identification, authentication, and authorization** to protect information assets.

2.5.1 Definition of Access Control

Access control is the **process of regulating who can access a system, resource, or information and what actions they are allowed to perform**. It determines **how access is granted, enforced, monitored, and revoked**.

Access controls are fundamental security techniques that determine authorized access to data, applications, systems, and physical assets. They encompass policies and mechanisms to verify identities, grant appropriate permissions, and enforce the principle of least privilege, ensuring entities only access what they need for their roles.

Access control protects:

- Files and folders
- Databases
- Applications
- Network resources
- Physical facilities (using electronic systems)

Access control ensures that users **cannot read, modify, delete, or execute resources** unless they are explicitly permitted to do so.

Real-life example:

In a company, employees use ID cards to enter the building. Only IT staff can enter the server room, while general employees are restricted from it.

2.5.2 Authentication Mechanism

Authentication mechanisms are used to **verify the identity of a user** before access is granted. Authentication is a **prerequisite to access control**.

Authentication mechanisms are commonly classified into the following types:

1. Knowledge-Based Authentication

This mechanism is based on **something the user knows**.

- Username and password
- PIN
- Security questions

Example: Logging into an email account using a username and password.

2. Possession-Based Authentication

This mechanism is based on **something the user has**.

- Smart cards
- One-Time Password (OTP) tokens
- Mobile phones (OTP via SMS or app)

Example: Using an ATM card along with a PIN.

3. Biometric-Based Authentication

This mechanism is based on **something the user is**.

- Fingerprints
- Face recognition
- Retina scan
- Voice recognition

Example: Unlocking a smartphone using fingerprint or face recognition.

4. Multi-Factor Authentication (MFA)

MFA combines **two or more authentication factors** to enhance security.

Example: Online banking login using password + OTP sent to mobile.

2.5.3 Access Control Principles

Access control principles guide the **design and implementation of secure access control systems**.

1. Principle of Least Privilege

Users should be given **only the minimum permissions required** to perform their tasks. This reduces the risk of misuse or accidental damage.

Example: A data entry operator can enter data but cannot delete database records.

2. Separation of Duties

Critical tasks are divided among multiple users so that **no single individual has complete control** over a process.

Example: One employee prepares a payment, while another approves it.

3. Need-to-Know Principle

Users are granted access **only to information necessary for their role**, even if they have high privileges.

Example: A manager can access reports related to their department only.

4. Default Deny

By default, **access is denied unless explicitly granted**. This minimizes unauthorized access.

Example: New employees have no system access until permissions are assigned.

5. Accountability

All access and actions must be **traceable to a specific user**, ensuring responsibility and auditability.

Example: System logs record which user modified a file.

2.5.4 Access Rights, Permissions, and Access Control Policies

Access Rights and Permissions

Access rights define **what actions a user can perform on a resource**. Common access rights include:

- Read
- Write
- Execute
- Modify
- Delete

Permissions are assigned to users or groups based on **roles, attributes, or security policies**.

Example:

A file permission may allow a user to read and write a document but not delete it.

2.5.4 Access rights and permission Access control policies

1. Discretionary access control (DAC)
2. Mandatory access control (MAC)
3. Role-based access control(RBAC)
4. Attribute-based access control (ABAC)

Access control is a fundamental security process that regulates who can access systems or information and what actions they are permitted to perform, such as reading, writing, or deleting resources. Below is a detailed breakdown of the four primary access control models as described in the sources.

1. Discretionary Access Control (DAC)

Discretionary Access Control is a model where the **owner of a resource determines who can access it** and what specific permissions they have.

- **Characteristics:** Access is managed through **Access Control Lists (ACLs)**, and the model is known for being flexible and user-friendly.
- **Advantages:** It is **easy to implement** and is particularly suitable for personal use or small systems.
- **Disadvantages:** Because control is decentralized, it is **vulnerable to malware** and misuse by insiders; it also lacks strong centralized control.
- **Real-life Example:** In operating systems like **Windows or Linux**, file owners have the discretion to decide which other users can read, write, or execute their specific files.

2. Mandatory Access Control (MAC)

Mandatory Access Control is a **strict, centralized model** where access decisions are not made by users but by the system itself based on predefined security policies.

- **Characteristics:** It relies on **security labels and classifications** (such as "Top Secret" or "Secret") assigned to both users and resources. Users are prohibited from changing access permissions themselves.
- **Advantages:** It provides **very high security** and is highly effective at preventing data leakage.
- **Disadvantages:** It is **complex to implement** and lacks the flexibility found in other models.
- **Real-life Example:** **Military and government systems** use MAC to ensure that classified documents are only accessible to individuals with the corresponding security clearance levels.

3. Role-Based Access Control (RBAC)

Role-Based Access Control shifts the focus from individual users to **organizational roles**. Permissions are assigned to roles, and users are then assigned to those roles.

- **Characteristics:** It simplifies access management by **linking permissions to job functions** rather than individual identities.
- **Advantages:** It is highly **scalable and efficient**, as it closely matches the actual structure of an organization.
- **Disadvantages:** It requires **careful and thorough role definition** and may be less flexible when dealing with unique or "one-off" access cases.
- **Real-life Example:** In a **hospital system**, a Doctor may have the role to access full patient records, a Nurse might have the role to update vitals, and a Receptionist has the role to only schedule appointments.

4. Attribute-Based Access Control (ABAC)

Attribute-Based Access Control is a sophisticated model that grants access based on a combination of **attributes related to users, resources, the environment, and specific actions**.

- **Attributes Used for Decisions:**
 - **User Attributes:** Role, department, or physical location.
 - **Resource Attributes:** File type or the sensitivity level of the data.
 - **Environmental Attributes:** The time of day, the specific device being used, or the IP address.
- **Detailed Insights:** Unlike RBAC, which is limited to roles, ABAC can use **environmental context** (like denying access if the user is logging in from an unknown IP address or outside of business hours) to make more granular access decisions,.

Summary Table of Access Control Models

Model	Decision Maker	Basis for Access	Primary Use Case
DAC	Resource Owner	User Identity & ACLs	Personal computers/Small systems,
MAC	System/Administrator	Security Labels & Clearances	Military/Government,
RBAC	Organization	Job Function (Role)	Corporate/Healthcare structures,
ABAC	Policy Engine	User, Resource, & Environment Attributes	Dynamic/Complex environments,

Below is a **very detailed, long-answer, exam-oriented explanation** of the four main **Access Control Models—DAC, MAC, RBAC, and ABAC**—written in **clear language, structured headings, point-wise paragraphs, advantages, limitations, and real-life examples**, suitable for **engineering and university exams**.

Access Control Models

Access control models define **how access permissions are assigned, enforced, and managed** in computer systems. They ensure that resources are accessed **only by authorized users** according to defined security policies.

1. Discretionary Access Control (DAC)

Definition

Discretionary Access Control (DAC) is an access control model in which **the owner of a resource has the authority to decide who can access it and what operations they can perform**.

In DAC, access rights are granted or revoked at the **discretion of the resource owner**.

How DAC Works

- Each resource has an owner
 - The owner assigns permissions to other users
 - Permissions are stored in **Access Control Lists (ACLs)**
-

Characteristics

- Owner-controlled access
 - Flexible and user-friendly
 - Decentralized permission management
-

Advantages

- Easy to implement and manage
 - Suitable for personal and small-scale systems
 - Supports file sharing
-

Limitations

- Vulnerable to malware and Trojan attacks
 - Weak protection against insider misuse
 - No strict centralized security enforcement
-

Real-Life Example

In Windows or Linux operating systems, file owners can decide who can **read, write, or execute** their files using file permissions.

2. Mandatory Access Control (MAC)

Definition

Mandatory Access Control (MAC) is a **strict access control model** where access decisions are made by the **system based on security labels**, not by individual users.

Users **cannot change access permissions**, even if they own the resource.

How MAC Works

- Each resource is assigned a **security classification** (e.g., Top Secret, Secret)
 - Each user is assigned a **security clearance**
 - Access is granted only if $\text{clearance} \geq \text{classification}$
-

Characteristics

- Centralized control
 - Highly secure
 - Rigid policy enforcement
-

Advantages

- Prevents data leakage
 - Strong protection against insider threats
 - High integrity and confidentiality
-

Limitations

- Complex to implement
 - Less flexible
 - Expensive
-

Real-Life Example

Military and government systems where classified information is accessible only to personnel with appropriate clearance levels.

3. Role-Based Access Control (RBAC)

Definition

Role-Based Access Control (RBAC) assigns access permissions based on **organizational roles** rather than individual users.

Users inherit permissions by being assigned to roles.

How RBAC Works

- Define roles (e.g., Admin, Manager, Employee)
 - Assign permissions to roles
 - Assign users to roles
-

Characteristics

- Role-centered
 - Simplifies permission management
 - Scalable
-

Advantages

- Easy to manage in large organizations
 - Supports separation of duties
 - Reduces administrative overhead
-

Limitations

- Role explosion if poorly designed
 - Less flexible for dynamic conditions
-

Real-Life Example

In a hospital system:

- Doctors can view and update patient records
 - Nurses can update vitals
 - Receptionists can manage appointments
-

4. Attribute-Based Access Control (ABAC)

Definition

Attribute-Based Access Control (ABAC) grants access based on **attributes of users, resources, actions, and environment**.

Access decisions are made dynamically using **policy rules**.

Types of Attributes

- User attributes (role, department, age)
 - Resource attributes (classification, owner)
 - Environmental attributes (time, location, device)
-

How ABAC Works

- Policies evaluate attributes
 - Access is granted only if all policy conditions are satisfied
-

Characteristics

- Fine-grained control
 - Highly flexible
 - Policy-driven
-

Advantages

- Ideal for cloud and distributed systems
 - Context-aware access
 - Reduces role explosion
-

Limitations

- Complex to configure
- Requires policy engines and attribute management

Real-Life Example

A cloud system allows access to confidential data **only if**:

- User is a manager
 - Access occurs during office hours
 - Login is from a company-approved device
-

Recommended Access Control Model for a Business System

For a business (enterprise) system, the best and most commonly recommended access control model is **Role-Based Access Control (RBAC)**.

Recommended Model: Role-Based Access Control (RBAC)

Justification

A business system typically includes **multiple departments, job functions, and hierarchical roles** such as managers, employees, HR staff, finance staff, and IT administrators. RBAC is best suited to handle these requirements efficiently and securely.

1. Aligns with Organizational Structure

Businesses operate based on **job roles and responsibilities**. RBAC assigns permissions to roles rather than individual users.

Example:

- HR role → access employee records
- Finance role → access payroll and accounting systems
- Manager role → approve reports and transactions

This mirrors real-world business workflows.

2. Simplifies Access Management

In large organizations, managing permissions individually (DAC) becomes complex and error-prone. RBAC allows administrators to:

- Create roles once
- Assign users to roles

When an employee changes roles, permissions change automatically.

Example:

When an employee is promoted from staff to manager, assigning the “Manager” role instantly updates access rights.

3. Supports Least Privilege and Separation of Duties

RBAC enforces:

- **Least privilege** by granting only role-specific permissions
- **Separation of duties** by preventing one user from holding conflicting roles

Example:

A user cannot both **create** and **approve** financial transactions, reducing fraud risk.

4. Scalable and Efficient

RBAC scales well as the business grows:

- New employees → assign existing roles
- New applications → map permissions to roles

This reduces administrative overhead compared to DAC or MAC.

5. Easier Compliance and Auditing

Business systems must comply with regulations (ISO 27001, SOX, GDPR). RBAC makes audits easier by clearly showing:

- Who has which role
- What permissions each role includes

Auditors can quickly verify access control policies.

6. Practical and Cost-Effective

Unlike MAC (which is rigid and expensive), RBAC provides **strong security with flexibility**, making it cost-effective for commercial environments.

Why Other Models Are Less Suitable

DAC (Discretionary Access Control)

- ✗ Too flexible and insecure for businesses
 - ✗ Users can share access unintentionally
 - ✓ Suitable only for small or personal systems
-

MAC (Mandatory Access Control)

- ✗ Very rigid and complex
 - ✗ Designed for military/government use
 - ✓ Overkill for most business systems
-

ABAC (Attribute-Based Access Control)

- ✓ Very powerful and flexible
- ✗ Complex to design and manage
- ✗ Higher implementation cost

Often used **along with RBAC** in advanced enterprise or cloud systems, but **not ideal as a standalone model** for typical businesses.

Conclusion: Role-Based Access Control (RBAC) is the **best access control model for business systems** because it:

- Matches business roles
- Is easy to manage and scale
- Supports security principles
- Simplifies compliance and audits

Best Practice:

Most modern business systems use **RBAC as the core model**, sometimes enhanced with **ABAC for context-aware controls**.

College system: recommend best access control model (DAC / MAC / RBAC / ABAC) and justify

Recommended model: RBAC (Role-Based Access Control)

Justification

- Natural fit for colleges: There are clear, stable role categories (student, faculty, admin, registrar, librarian, IT staff). Assigning permissions to roles maps directly to organizational responsibilities.
- Scalability: Adding/removing users is simple — assign or revoke roles rather than edit ACLs for each user.
- Least privilege & separation of duties: Roles can be crafted to limit privileges (e.g., students can't modify grades). Multiple roles can be combined for staff with multiple responsibilities.

- Manageability: Role changes (promotion, graduation) are easier to handle than updating thousands of individual ACL entries (which DAC would require).
- Auditing & compliance: Easier to audit which role has what rights, and to produce role-based logs for compliance.

Example mapping for college

- Student role: register for courses, access course materials, submit assignments, view own grades
- Faculty role: create course content, grade assignments, view student lists
- Registrar role: edit student records, generate transcripts, approve registrations
- Librarian role: manage library database, issue/return records
- IT role: manage accounts and infrastructure (limited to system operations, not academic records unless necessary)

When other models might be used:

- DAC could be used for personal file sharing among faculty (owner-controlled sharing), but not for core institutional access control.
- MAC is appropriate only if the institution requires classification levels (e.g., research labs with classified projects).
- ABAC (Attribute-Based) can handle dynamic context (time of day, location), and is powerful — but it is more complex to design and manage. ABAC could be used in addition for special cases (e.g., access allowed only from campus network during business hours).

Below is a **clear, exam-oriented explanation of when each access control model (DAC, MAC, RBAC, ABAC) is used in real life, with practical cases and justification.**

Real-Life Use Cases of Access Control Models

Different access control models are chosen based on **security level, flexibility, environment, and organizational needs**. No single model fits all situations.

Below are **case-study-based justifications** showing **why each access control model (DAC, MAC, RBAC, ABAC) is used in real-life scenarios**, written in a **descriptive, exam-ready format**.

Case-Study-Based Justification of Access Control Models

Case Study 1: Discretionary Access Control (DAC)

Scenario: Small Software Startup

A small software startup has 15 employees working on shared documents, source code, and project reports. Each employee owns certain files and often needs to share them with teammates.

Why DAC Is Used

- File owners decide who can access their files
- Employees frequently share and modify documents
- Security requirements are moderate
- Administrative overhead must be minimal

Implementation

Employees use a shared file server where file owners set **read/write permissions** for other team members.

Justification

DAC is suitable because the environment requires **flexibility and ease of sharing**, not strict centralized security.

Case Study 2: Mandatory Access Control (MAC)

Scenario: Government Defense Organization

A government defense agency handles classified intelligence data labeled as **Top Secret, Secret, and Confidential**. Any unauthorized disclosure could threaten national security.

Why MAC Is Used

- Data sensitivity is extremely high

- Access must follow strict government policies
- Users must not modify permissions
- Strong protection against insider threats

Implementation

Documents are labeled by classification. Users receive clearance levels. Access is granted only when clearance $\geq$ classification.

Justification

MAC ensures **maximum confidentiality and integrity**, making it ideal for military and government environments.

Case Study 3: Role-Based Access Control (RBAC)

Scenario: Banking System

A bank has thousands of employees across departments such as teller services, loans, customer support, and management.

Why RBAC Is Used

- Employees perform role-specific tasks
- Permissions depend on job responsibilities
- Regulatory compliance is required
- User turnover is high

Implementation

Roles such as Teller, Loan Officer, Manager, and Auditor are defined. Users inherit permissions by role assignment.

Justification

RBAC simplifies management, supports **least privilege and separation of duties**, and enables easier audits.

Case Study 4: Attribute-Based Access Control (ABAC)

Scenario: Cloud-Based Enterprise with Remote Employees

A multinational company uses cloud services with employees working from different locations, devices, and time zones.

Why ABAC Is Used

- Access depends on time, location, device, and user attributes
- Traditional role-based rules are insufficient
- High need for dynamic access decisions

Implementation

Policies grant access only if:

- User is in the correct department
- Login occurs during office hours
- Device is company-approved
- Location is trusted

Justification

ABAC provides **fine-grained, context-aware control**, making it ideal for cloud and remote environments.

Case Study 5: Hybrid Model (RBAC + ABAC)

Scenario: Modern Enterprise IT System

A large enterprise uses:

- Office-based employees
- Remote workers
- Cloud-hosted applications

Why Hybrid Model Is Used

- RBAC handles job roles
- ABAC adds context-based restrictions

Implementation

An employee has a “Manager” role (RBAC) but can access sensitive data only:

- From office network
- During working hours
- Using company laptop (ABAC)

Justification

The hybrid approach balances **manageability and security**, which is common in real-world enterprise systems.

Comparative Case Summary

Case Study	Model Used	Key Reason
Startup file sharing	DAC	Flexibility
Defense agency	MAC	Maximum security
Bank system	RBAC	Role-based efficiency
Cloud enterprise	ABAC	Context-aware access
Modern enterprise	RBAC + ABAC	Balanced control

Conclusion

Case-study analysis shows that **each access control model is best suited to specific real-life conditions**:

- **DAC** for flexibility
- **MAC** for high security
- **RBAC** for structured organizations
- **ABAC** for dynamic environments

In practice, **hybrid models** are increasingly adopted to meet modern security demands.

INSHORT

1. Discretionary Access Control (DAC) – When Flexibility Is Needed

Used When:

- Users need **freedom to share resources**

- Environment is **small or non-critical**
- Centralized security enforcement is not strict

Real-Life Use Cases:

- **Personal computers and laptops**
- **Small offices or startups**
- **File sharing in Windows/Linux systems**
- **Academic environments**

Example:

A teacher shares lecture notes with selected students by changing file permissions.

Why DAC Fits Here:

- ✓ Easy to use
 - ✓ User-controlled sharing
 - ✗ Not suitable for high-security systems
-

2. Mandatory Access Control (MAC) – When Maximum Security Is Required

Used When:

- Data is **highly sensitive or classified**
- Security policies must be **strictly enforced**
- Users must not change permissions

Real-Life Use Cases:

- **Military and defense systems**
- **Government intelligence agencies**
- **Nuclear facilities**
- **Critical infrastructure systems**

Example:

A defense officer can access “Top Secret” files only if they have the required clearance.

Why MAC Fits Here:

- ✓ Strong confidentiality and integrity
- ✓ Prevents insider threats
- ✗ Rigid and expensive

3. Role-Based Access Control (RBAC) – When Organizational Roles Matter

Used When:

- Access depends on **job roles**
- Organization has **many users**
- Compliance and auditing are important

Real-Life Use Cases:

- **Business and enterprise systems**
- **Banks and financial institutions**
- **Hospitals**
- **Universities**
- **ERP systems (SAP, Oracle)**

Example:

In a hospital:

- Doctors access patient records
- Nurses update vitals
- Admin staff manage billing

Why RBAC Fits Here:

- ✓ Easy management
- ✓ Scalable
- ✓ Supports least privilege

4. Attribute-Based Access Control (ABAC) – When Context Matters

Used When:

- Access decisions depend on **context**
- Systems are **dynamic and distributed**
- Fine-grained control is required

Real-Life Use Cases:

- Cloud computing platforms (AWS, Azure)
- Remote work environments
- IoT systems
- Zero-Trust security architectures

Example:

An employee can access company data **only during office hours, from a company laptop, and from a trusted location.**

Why ABAC Fits Here:

- ✓ Highly flexible
 - ✓ Context-aware
 - ✓ Ideal for modern systems
 - ✗ Complex to manage
-

Hybrid Use in Real Life (Very Common)

In practice, **organizations rarely use only one model.**

Common Combinations:

- **RBAC + ABAC** → Enterprises & cloud systems
- **MAC + RBAC** → Government organizations
- **DAC + RBAC** → Universities and offices

Example:

A company:

- Uses **RBAC** for job roles
 - Uses **ABAC** to restrict access based on time, location, and device
-

Quick Summary Table

Model	Best Used In	Key Reason
DAC	Personal / Small systems	Flexibility
MAC	Military / Government	Maximum security

RBAC	Business / Enterprise	Role-based management
ABAC	Cloud / Remote / IoT	Context-aware access

Conclusion

Each access control model serves a **specific real-life purpose**:

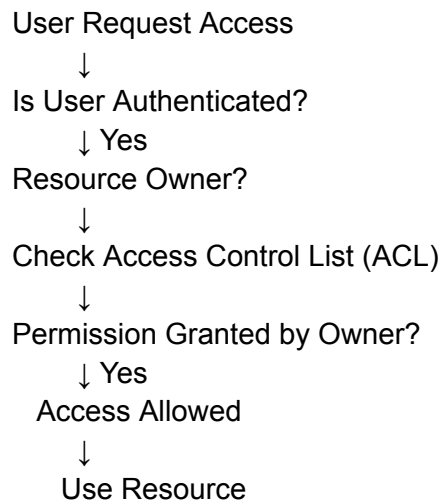
- **DAC** → Flexibility
- **MAC** → High security
- **RBAC** → Organizational efficiency
- **ABAC** → Dynamic, context-aware security

Choosing the right model depends on **risk level, scale, and system complexity**.

Below are **clear, exam-friendly flow diagrams** for **each access control model (DAC, MAC, RBAC, ABAC)**.

They are written in **simple ASCII format**, which is commonly accepted in **written exams**.

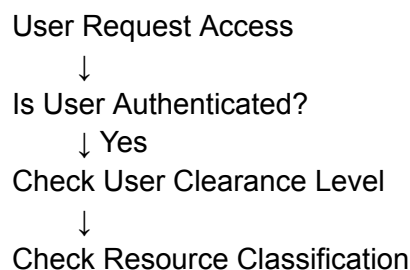
1. Discretionary Access Control (DAC) – Flow Diagram

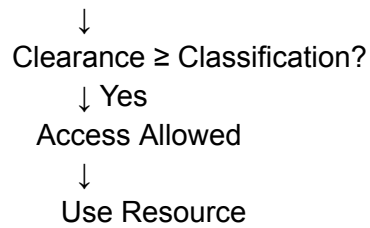


Explanation

- Resource owner decides access
 - ACL stores permissions
 - Flexible but less secure
-

2. Mandatory Access Control (MAC) – Flow Diagram

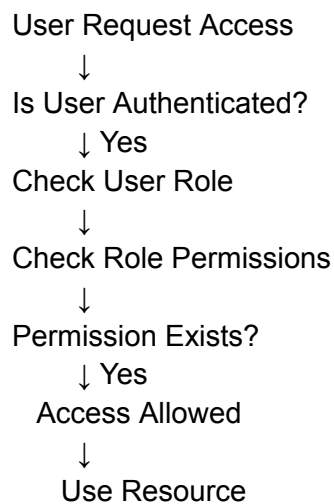




Explanation

- Central authority controls access
 - Users cannot change permissions
 - Very high security
-

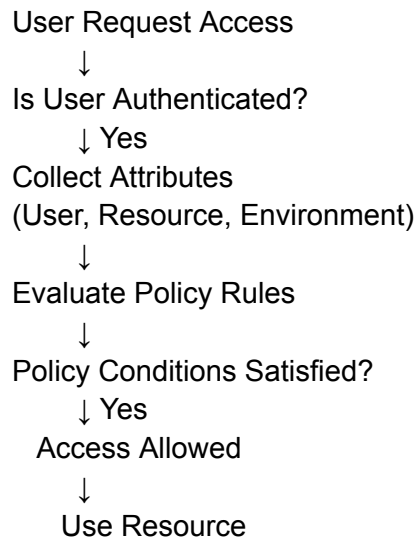
3. Role-Based Access Control (RBAC) – Flow Diagram



Explanation

- Permissions linked to roles
 - Users inherit permissions via roles
 - Ideal for organizations
-

4. Attribute-Based Access Control (ABAC) – Flow Diagram



Explanation

- Decision based on attributes
 - Context-aware and dynamic
 - Used in cloud and modern systems
-

5. Comparative Visual Summary (Optional for Exams)

DAC → Owner decides
MAC → System decides (labels)
RBAC → Role decides
ABAC → Policy + attributes decide
